WhatsApp + Shopify AI Support Bot

Project overview: architecture, features, setup, costs and status

Tussles Store (Lahore) · state on 20 October 2026

In one paragraph. Customers write to the store on WhatsApp in English, Roman Urdu, Urdu script or Punjabi, by text or voice note. The bot answers questions about watches (price, stock, colours, sizes, pictures) from the **live Shopify store**, and questions about delivery, returns, privacy and FAQ from the **store's own policies and pages**. It never changes anything in the store (read-only), never invents a fact, refuses attacks and private data requests, and hands the customer to a person (email alert) when needed. The owner can answer first from the WhatsApp Business app; the bot then stays silent.

Item	Value
Store	Tussles Store, Shopify (dev store now; the same store goes live)
Channel	WhatsApp Business Platform (Meta Cloud API). Test number now; the store's own number later
Code	Python 3.12, FastAPI, LangGraph, PostgreSQL + pgvector. GitHub repo, CI on every push
AI	Groq: openai/gpt-oss-120b (understand + write), whisper-large-v3 (voice). Local embeddings
Status	Phases 1 to 12 built and tested (2,139 automated tests pass). Live tests on the phone passed. Left: real number, paid AI plan, AWS deploy, launch checklist
Design source	Shopify_WhatsApp_AI_Support_Bot_Master_Blueprint_v2.pdf (14 phases, 25 test cases)

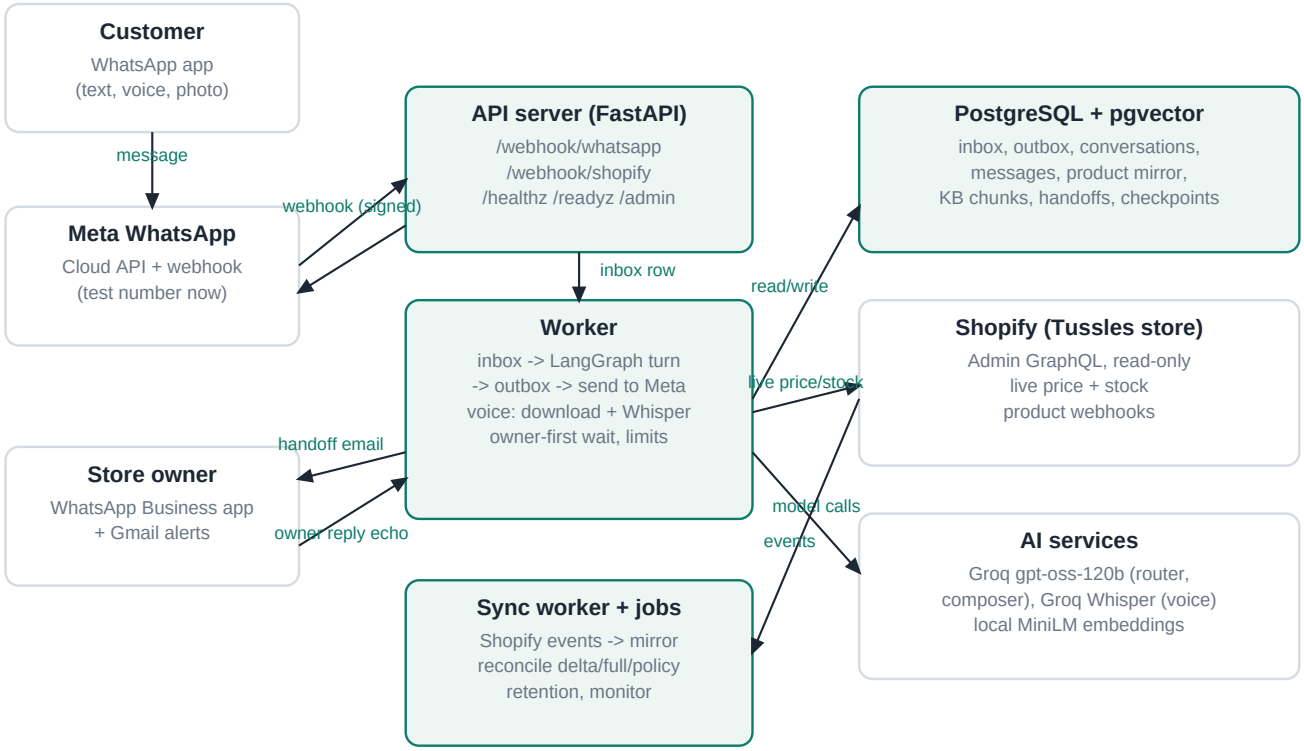
1. What the bot can do

Area	Feature	Example
Products	Price, stock, colours and sizes, live from Shopify (never from memory)	"Rolex Submariner, 16610 ki price?" -> Rs 12,999
	Search with typos, model codes and brand short forms (RLX = Rolex, CRTR = Cartier, OMGA = Omega, PP = Patek Philippe, AP = Audemars Piguet)	"Rolex Submariner" -> lists all 5 Submariner models and asks which one
	Several close matches: one short question, never a guess	"Aap ka matlab kaun sa hai: A ya B?"
	Budget / colour / size only: a short list and a question	"10k ke andar koi watch hai?" -> 5 watches under Rs 10,000
	Product pictures (up to 3), converted to JPEG for WhatsApp	"Iski picture bhejo"
Store info	Delivery, returns, privacy, terms, FAQ answered from the store's policies and pages (RAG)	"Delivery kitne din mein?" -> "Lahore, 2 working days"
	Says "not confirmed" and offers the team when the documents do not hold the answer	
Languages	English, Roman Urdu, Urdu script, mixed; reply in the customer's language (Urdu script gets Roman Urdu)	"English mein bataein" switches the language
Voice	Voice notes transcribed (Whisper); unclear voice -> "please repeat or type"; misheard rude words never punish the customer	Urdu, English, Punjabi (Punjabi weaker)

Area	Feature	Example
People	Handoff to the team (complaint, "insaan se baat", unclear 3 times) with an email alert; bot pauses for that customer	
	Owner first (built, off until the real number): the owner answers from the Business app, the bot waits 3 minutes in owner hours and stays silent after an owner reply	
Safety	Refuses prompt injection, secret / API key requests, other customers' data, discount tricks; card numbers masked	3 attacks -> one cool-down notice, then silence
	Rate limits, daily cost budget, kill switch, admin API (off by default)	
Privacy	Phone numbers hashed in logs; data deleted after 180/90 days; "delete my data" script	

2. Architecture

The system is a few small processes around one PostgreSQL database. WhatsApp and Shopify talk to the API server through signed webhooks; the API only stores what came in and answers 200 at once. The worker does the real work.



Part	Runs as	Job
API server	uvicorn app.main:app	Receives WhatsApp and Shopify webhooks (signature checked), stores them, health checks, admin API (off by default), /dev endpoints (local only)
Worker	python -m app.worker	Takes one inbound message at a time: owner check, rate limit, pause, budget, card check, voice transcription, then the LangGraph turn; queues replies in the outbox and sends them to Meta with retries
Sync worker	python -m app.sync.worker	Handles Shopify product/inventory webhooks: refetches the product and updates the mirror; dead letter after 5 tries
Jobs	python -m app.jobs.reconcile / retention	delta (every few minutes), full (nightly), policy (hourly: policies and pages into the knowledge base), retention (daily cleanup)

Part	Runs as	Job
Database	Docker: pgvector/pgvector:pg16	All state: inbox, outbox, conversations, messages, product mirror, knowledge-base chunks with vectors, handoffs, security events, sync runs, LangGraph checkpoints
Tunnel (dev)	ngrok static domain	Public HTTPS address for the webhooks while the bot runs on the laptop. AWS replaces it in Phase 13

2.1 What happens to one message

1. Meta posts the message to **/webhook/whatsapp**. The signature (app secret) is checked; the message is stored in **whatsapp_inbox** (a duplicate delivery is ignored by its message id).
2. The **worker** claims the oldest pending message. With owner-first on, a new message waits 3 minutes in owner hours.
3. Cheap checks first, no AI cost: kill switch, owner answering this customer, rate limit / cool-down, handoff pause, daily budget, card number warning.
4. Voice note: fresh media URL from Meta, download, ffmpeg to WAV, Whisper, then a confidence / language gate.
5. The **LangGraph** turn (below) produces the reply text or pictures.
6. Replies go to **whatsapp_outbox**; the sender delivers them (pictures: link, then upload, then a text link).
7. The worker logs one **turn_done** line: intent, tools, errors, fallback, model calls and tokens (no customer words).

2.2 The LangGraph turn (the bot's brain)

Step	What it does
normalize + detect language	Clean the text (length limit, hidden characters), cheap language hint
router (AI call 1)	Reads prompts/router.md and returns JSON: language, intent (17 intents), product words, colour, size, price limit, English search query for policies. Invalid JSON gets one repair, then a safe 'unclear'. The price limit is checked by a rule-based parser
permission gate	Decides the branch and which tools this intent may use. Attacks go to 'refuse' and are recorded
tools	product_info: search_products, get_price, check_stock, get_colors_sizes, get_specs, get_product_images (live Shopify). policy_rag: search_policy over the knowledge base. Max tool calls per turn
clarify / refuse / handoff	Fixed templates (prompts/*.yaml): one short question, a polite refusal, or 'team will contact you' + email alert. No AI cost
composer (AI call 2)	Writes the reply from the tool results only (prompts/composer.md), in the reply language
validator	Rejects a reply that states a price, stock, colour or policy fact the tools did not give, or leaks a secret. One retry, then a safe fallback + handoff
finish	Saves the conversation state (language, current product, pause, last 6 lines of history). A Postgres checkpointer keeps every step for debugging

2.3 Other flows

Flow	How it works
Voice note	Media id -> fresh download link from Meta (links expire after a few minutes, so it is always fetched fresh) -> size/length limits (90 s, 5 MB) -> ffmpeg -> Groq Whisper -> language and confidence gate -> same graph as text. Unclear audio gets 'please repeat or type'
Pictures	get_product_images returns up to 3 Shopify image URLs. WhatsApp only accepts JPEG/PNG, so WebP is converted with Pillow and uploaded to Meta when the link fails; last fallback is a text link
Policy questions	Router writes an English search query -> pgvector cosine search over kb_chunks (plus pg_trgm keyword match) -> top 5 chunks above score 0.40 -> composer answers only from them, else 'not confirmed + team'

Flow	How it works
Shopify change	Shopify webhook (HMAC checked) -> sync_events row -> sync worker refetches the product by GraphQL -> products_mirror / variants_mirror / product_images updated. Reconcile jobs catch anything missed
Owner first	Owner replies from the WhatsApp Business app arrive as 'smb_message_echoes' webhooks -> stored as an outgoing message -> that customer gets no bot reply for 60 minutes. 'python -m app.owner away/back' switches it by hand
Handoff	Template reply to the customer, a handoffs row, an email (Gmail SMTP) with the reason and a short summary; bot paused 12 hours for that customer

3. Data: what is stored where

Table	Holds
whatsapp_inbox	Every inbound WhatsApp message (unique wamid = no duplicate replies), status pending / done / failed
whatsapp_outbox	Every reply to send, with retries and Meta's message id
conversations	One row per customer (phone hash + number): state JSON (language, current product, pause, owner_active_until), last inbound time
messages	Short history of in/out texts (90 days)
products_mirror / variants_mirror / product_images	Copy of the Shopify catalogue for fast search (titles, handles, tags, options, price, stock, images). Prices/stock are also re-checked live
kb_documents / kb_chunks	Store policies and pages, split into chunks with a 384-number vector each (embedding)
handoffs	Handoff reason, summary, notified yes/no
security_events	Injection / secret / data-request attempts (no full text), used for the cool-down
sync_events / sync_runs	Shopify webhook queue and reconcile job history
runtime_flags	Switches set at run time: kill switch, owner_away
LangGraph checkpoint tables	Step-by-step state of each turn (7 days), created by AsyncPostgresSaver

*Database migrations are managed with **alembic** (alembic upgrade head). Postgres extensions: **vector** (pgvector) and **pg_trgm**.*

4. Technology used

Layer	Tools	Why
Language / packaging	Python 3.12, uv (fast installer and lock file), hatchling	One lock file, same versions everywhere
Web	FastAPI, uvicorn, httpx, tenacity (retries)	Async webhooks, fast 200 replies
AI orchestration	LangGraph 1.x, LangChain core, langchain-openai (any OpenAI-compatible host), langgraph-checkpoint-postgres	Fixed graph = predictable, testable steps; provider can change by .env
Models	Groq openai/gpt-oss-120b; Groq whisper-large-v3; sentence-transformers paraphrase-multilingual-MiniLM-L12-v2 (local, free); Ollama gpt-oss for evals	Cheap, fast, multilingual
Data	PostgreSQL 16 + pgvector + pg_trgm, SQLAlchemy 2 async, asyncpg, psycopg 3, alembic	One database for queue, catalogue, vectors and checkpoints
Validation	pydantic 2, pydantic-settings	Every model output and setting is typed and checked
Content	BeautifulSoup (policy HTML to text), Pillow (images), ffmpeg (audio), PyYAML (templates, aliases)	

Layer	Tools	Why
Monitoring	LangSmith traces (customer text hidden in production), JSON logs, app.monitor	See every step, cost and failure
Quality	pytest (+asyncio, cov, respx), ruff, mypy, pip-audit, pre-commit, GitHub Actions CI	
Dev infra	Docker Compose (pgvector/pgvector:pg16), ngrok static domain	Same database as production; public URL for webhooks

5. How the Shopify app is made

The bot reads the store through a **custom app made in the Shopify Dev Dashboard**. It is not public and not on the App Store; it is installed only on Tussles Store. The bot has **read-only** access: it cannot change products, prices, orders or customers.

1. Open **dev.shopify.com** (Dev Dashboard) with the store owner's / partner account and **Create app** (name e.g. 'ws-bot').
2. In the app's version, set the **Admin API scopes** (read only): **read_products**, **read_inventory**, **read_legal_policies**, **read_online_store_pages**. Nothing with 'write_', no orders, no customers.
3. Release the version and **install** the app on the store.
4. Copy **Client ID** and **Client secret** from the app's settings into .env (SHOPIFY_CLIENT_ID, SHOPIFY_CLIENT_SECRET) with SHOPIFY_SHOP_DOMAIN = the store's myshopify.com domain and SHOPIFY_API_VERSION = 2026-10.
5. Auth mode **client_credentials**: the bot asks Shopify for an access token with the ID + secret; the token lives 24 hours and the bot renews it on its own. (SHOPIFY_OFFLINE_TOKEN is only for the older install flow.)
6. Register webhooks: **uv run python scripts/register_webhooks.py** with SHOPIFY_WEBHOOK_BASE_URL = the public HTTPS address. Topics: products/create, products/update, products/delete, inventory_levels/update. Each webhook is signed; the API checks the HMAC with the client secret.
7. First full import: **python -m app.jobs.reconcile full**, then **policy** for policies and pages; after that the --loop mode keeps it fresh (delta 5 min, policy 60 min, full 24 h).

What the store team must keep in Shopify (the bot only knows what is written there): correct price and stock; colour / size as variant options; case size and strap in the description or a metafield; warranty (machine only), delivery (Lahore, 2 working days), returns and FAQ written in the policies or an FAQ page. After a big change run **reconcile policy** / **full** once.

6. How the WhatsApp number is attached (Meta / Facebook)

WhatsApp messages come through Meta's **WhatsApp Business Platform (Cloud API)**. The pieces are: a Meta **Business portfolio** (business.facebook.com), a **developer app** (developers.facebook.com), a **WhatsApp Business Account (WABA)** and a **phone number** inside it.

6.1 Done (test setup)

1. Developer app **ws-bot** created (type Business) and the **WhatsApp** product added. Meta gives a free **test number** and up to 5 verified recipient numbers.
2. A **System User** ('Wsbot', admin) in Business settings, given the app and the WABA, and a **permanent token** with **whatsapp_business_messaging** and **whatsapp_business_management** -> WHATSAPP_ACCESS_TOKEN. (The temporary 24-hour token is not used.)
3. Phone number ID -> WHATSAPP_PHONE_NUMBER_ID; App secret (App settings > Basic) -> WHATSAPP_APP_SECRET, used to check the webhook signature; a random WHATSAPP_VERIFY_TOKEN.
4. Webhook in the app: callback URL **https://<public address>/webhook/whatsapp** + the verify token; field **messages** subscribed. The WABA is subscribed to the app (subscribed_apps).

6.2 Still to do before launch (P2 to P7)

Step	What	Who
P2 Business verification	Business settings > Security centre: legal name, address, documents (registration / utility bill / tax). Raises limits and allows the display name	Client documents
P3 Billing	Add a payment method in Billing (WhatsApp Manager). Needed: after 1,000 free service conversations a month Meta stops delivery without it	Client card
P5 Real number	The store's existing WhatsApp Business number joins via **Coexistence** (Embedded Signup with the 'connect existing WhatsApp Business app' option, scan QR from the phone). The owner keeps using the Business app; chats sync. Then subscribe webhook field **smb_message_echoes** so owner replies reach the bot	
	Display name 'Tussles Store' approved by Meta; profile picture, about, address	
P6 Live mode	App switched from Development to Live (needs a privacy policy URL)	
P7 Phone test	All main tests on the real number, then OWNER_FIRST=true	

Coexistence note: the number stays in the phone's WhatsApp Business app and also works with the API. Messages the owner sends from the phone are reported to the bot as echoes, which is what makes 'owner first' work. Some app features (e.g. broadcast lists, disappearing messages) are limited under Coexistence.

7. Other outside services

Service	Used for	Setup
Groq (console.groq.com)	Router + composer (gpt-oss-120b), voice (whisper-large-v3)	API key -> LLM_API_KEY and STT_API_KEY; LLM_PROVIDER=groq. Free tier now (rate limited). Paid Developer tier is 'temporarily unavailable'; fallback hosts below
Gmail	Handoff email alerts	2-step verification on, then an **App password** (16 letters, spaces removed) -> SMTP_PASSWORD; smtp.gmail.com : 587. Test: python -m app.notifier --test
LangSmith	Traces of each turn	API key; LANGSMITH_HIDE_TEXT=true in production
Ollama	Offline evaluation of the router	Local or Ollama cloud model gpt-oss:120b-cloud
ngrok	Public HTTPS address in development	Free static domain; 'ngrok http --url=<domain> 8000'
GitHub	Code, CI (tests, lint, type check, pip-audit), release tags per phase	Secrets never committed (.env is git-ignored)
AWS (Phase 13)	Production hosting	Planned: containers for API / worker / sync / jobs, managed Postgres with pgvector, secrets manager, HTTPS

8. Configuration (.env)

All settings come from a **.env** file (never in Git; .env.example lists every name with a comment). Main groups:

Group	Names
App	APP_ENV, LOG_LEVEL, KILL_SWITCH, ADMIN_API_KEY, ADMIN_ENDPOINTS_ENABLED, PHONE_HASH_PEPPER
Database	DATABASE_URL
Shopify	SHOPIFY_SHOP_DOMAIN, SHOPIFY_API_VERSION, SHOPIFY_CLIENT_ID, SHOPIFY_CLIENT_SECRET, SHOPIFY_AUTH_MODE, SHOPIFY_WEBHOOK_BASE_URL, RECONCILE_*_MINUTES/HOURS, LOW_STOCK_THRESHOLD, SHOW_EXACT_STOCK, LIVE_STOCK_CACHE_SECONDS, ROUTER_CACHE_SECONDS
WhatsApp	META_GRAPH_VERSION, WHATSAPP_PHONE_NUMBER_ID, WHATSAPP_ACCESS_TOKEN, WHATSAPP_APP_SECRET, WHATSAPP_VERIFY_TOKEN, MAX_AUDIO_SECONDS, MAX_AUDIO_BYTES
LLM	LLM_PROVIDER, ROUTER_MODEL, COMPOSER_MODEL, LLM_API_KEY, LLM_TIMEOUT_SECONDS, TURN_BUDGET_SECONDS; eval: OLLAMA_*, EVAL_ROUTER_MODEL
Speech	STT_PROVIDER, STT_API_KEY, STT_MODEL, STT_MIN_CONFIDENCE, STT_MIN_WORDS, STT_ALLOWED_LANGUAGES, STT_LANGUAGE_HINT, STT_RETRY_LANGUAGE, STT_PROMPT
RAG	EMBEDDING_PROVIDER, EMBEDDING_MODEL, RAG_TOP_K (5), RAG_MIN_SCORE (0.40)
Language	URDU_SCRIPT_REPLIES, LANGUAGE_STICKY_MESSAGES, DEBUG_REPLY
Limits / cost	RATE_LIMIT_PER_MINUTE/HOUR/DAY (8/60/200), DAILY_COST_BUDGET, COST_PER_TEXT_MESSAGE, COST_PER_VOICE_MESSAGE
Retention (days)	CONVERSATION 180, MESSAGE 90, QUEUE 30, HANDOFF 90, SECURITY 365, SYNC 14, CHECKPOINT 7
Handoff	HANDOFF_CHANNEL (email), HANDOFF_TARGET, HANDOFF_PAUSE_HOURS (12), BUSINESS_HOURS, BUSINESS_UTC_OFFSET (+05:00), SMTP_*
Owner first	OWNER_FIRST, OWNER_WAIT_MINUTES (3), OWNER_HOURS, OWNER_REPLY_PAUSE_MINUTES (60)
Tracing	LANGSMITH_TRACING, LANGSMITH_API_KEY, LANGSMITH_PROJECT, LANGSMITH_HIDE_TEXT

9. Running it (development)

```
uv sync                                  # install packages from the lock file
docker compose up -d db                  # Postgres + pgvector on localhost:5432
uv run alembic upgrade head              # create / update tables
uv run python -m app.jobs.reconcile full  # import products   (then: reconcile policy)
uv run uvicorn app.main:app --reload --app-dir src   # API on :8000
uv run python -m app.worker              # WhatsApp worker
uv run python -m app.sync.worker         # Shopify webhook worker
uv run python -m app.jobs.reconcile --loop          # keeps the mirror fresh
ngrok http --url=<static-domain> 8000    # public address for webhooks
uv run python scripts/register_webhooks.py          # Shopify webhooks (once)
```

Task	Command
Tests / lint	uv run pytest -q | uv run ruff check . && uv run ruff format --check . | uv run mypy src
Health / monitor	GET /health, /ready | uv run python -m app.monitor
Kill switch	KILL_SWITCH=true (or the runtime flag) -> every customer gets the 'team will reply' message, no AI

Task	Command
Owner away / back	uv run python -m app.owner away | back | status
Email test	uv run python -m app.notifier --test
Delete a customer	uv run python scripts/delete_customer.py <number>
Daily cleanup	uv run python -m app.jobs.retention
Chat without WhatsApp	/dev endpoints (APP_ENV=local only) and the chat script for manual tests

10. Safety, privacy and limits

- **Read-only by design**: the Shopify app has only read scopes; the bot has no tool that writes anything.
- **Signed webhooks**: Meta (X-Hub-Signature-256, app secret) and Shopify (HMAC, client secret); unsigned calls are rejected.
- **Facts only from tools**: the validator blocks any price, stock, colour or policy fact not returned by a tool this turn.
- **Injection scan** before the model (ignore-instructions, new persona, secret requests, other customers' data, fake discounts) in English and Roman Urdu; 3 events -> cool-down. The store's own ToS text is not flagged.
- **Secrets**: only in .env / the secrets manager, never in prompts, logs or Git. Admin API off unless a 24+ character key is set.
- **Privacy**: phone numbers hashed (pepper) in logs and traces; card numbers detected and masked; customer data deleted at once on request; retention job removes old data.
- **Cost guards**: rate limits per customer, daily budget (alarm at 80 %, handoff-only at 100 %), turn time budget 45 s, router cache for repeated ad texts (ROUTER_CACHE_SECONDS), max tool calls per turn.
- **Reliability**: inbox/outbox queues, retries with backoff, dead letters, duplicate protection by message id, safe fallback reply when the model fails.

11. Testing and measured quality

2,139 automated tests (unit + database + graph) run on every push. Separate evaluation jobs use the real model:

Check	Result
Golden test set (blueprint cases)	65 / 65 pass
Natural customer messages	0 wrong facts
Tool choice accuracy	99.5 %
Language detection (router)	98.6 %
Policy retrieval	hit rate 97.5 %, correct 'don't know' 70 %
Security attacks	46 / 46 blocked
Duplicate replies	0
Latency (text)	p50 3.5 s, p95 5.7 s
Live phone tests	Manual WhatsApp tests incl. voice with expired media link (B12) and phone-number security (B8): pass

12. Costs

Three kinds of running cost: the AI models (per message), Meta (per message, after a free allowance) and hosting. Figures are list prices checked in October 2026; the real cost per message is read from the worker's **turn_done** logs (llm_calls, tokens_in, tokens_out) and then put into COST_PER_TEXT_MESSAGE / COST_PER_VOICE_MESSAGE.

12.1 AI models

Item	Price (USD)	Per customer message (estimate)
Groq gpt-oss-120b	$0.15 per 1M input tokens, $0.60 per 1M output	2 calls, about 4,500 tokens in + 500 out = about $0.001 (0.3 PKR)
Groq whisper-large-v3	about $0.11 per hour of audio (minimum 10 s billed)	20 s voice note = about $0.0006
Embeddings	Free (local model on the server)	0
Templates (clarify, refuse, handoff)	No AI call	0
Router cache (ads)	Same pre-filled ad text answered from cache	saves the router call

Rough monthly AI cost: 1,000 messages/day -> about $30/month; 300/day -> about $10/month.

If Groq paid tier stays unavailable	Price per 1M tokens (in / out)	Note
DeepInfra gpt-oss-120b	$0.037 / $0.17	Same model, cheaper; only .env change (OpenAI-compatible)
Amazon Bedrock gpt-oss-120b	$0.15 / $0.60	Same model; fits with AWS hosting / credits
OpenAI GPT-4.1 mini	about 2.5x the Groq cost	Different model, needs re-evaluation
Voice: OpenAI gpt-4o-mini-transcribe	$0.003 per minute	Fallback for Whisper

12.2 Meta (WhatsApp)

- Customer writes first and the bot replies within 24 hours = a **service** message. From 1 October 2026 Meta charges service messages after **1,000 free per month** (per Meta's Pakistan rate card). Without a payment method Meta stops delivery after the free 1,000, so billing (P3) is needed before launch.
- **Free Entry Point**: when a customer comes from a **Click-to-WhatsApp ad** (Facebook / Instagram ad with a WhatsApp button), the messages in that free-entry-point window are not charged (72 hours on the API; Meta's page mentions up to 7 days for replies from the phone app). Source: developers.facebook.com/documentation/business-messaging/whatsapp/pricing.
- The bot sends **no template / marketing** messages, so no marketing or utility fees.
- Number, API and webhooks themselves are free.

12.3 Hosting

- Development: the laptop + Docker + ngrok free = 0.
- Production (Phase 13, AWS): one small container service for API / workers / jobs + managed Postgres with pgvector + secrets. Exact size and price decided in Phase 13; AWS credits can cover the first months.

13. Client decisions (answered October 2026)

Topic	Decision
Number	The store's existing WhatsApp Business number (via Coexistence)
Payment card	One card for Meta, Groq and AWS; Groq paid plan yes
Owner	Both: owner hours and manual on/off. Active 11:00 to 23:00, no fixed timing
Handoff	Email to Gmail; team replies within 2 hours; the developer checks the alerts at first
Products	Brand short forms correct; show all Submariner models and ask; same model = only the strap differs; case sizes for some watches; smart watches yes; straps for some models
Policies	Delivery Lahore only; warranty on the machine only; no instalments
Language	Reply in Roman Urdu; Punjabi voice that is unclear -> ask to repeat
Data	History kept 6 months; delete at once on request; duplicate-reply risk accepted
Stores	Tussles Store stays separate from Suparnova

14. Status and what is left

Phase	Content	State
0-2	Repo, settings, database, Shopify read client, mirror + webhooks + reconcile	Done
3-4	WhatsApp webhook, inbox/outbox, sender, product search	Done
5-7	Router, LangGraph, tools, composer, validator, languages	Done
8	Policy RAG (embeddings, pgvector)	Done
9	Voice notes (Whisper)	Done
10	Security, rate limits, handoff, retention	Done
11	Evaluations and monitoring	Done
12	Hardening, owner first, router cache, token counting	Done
13	AWS deployment	Next
14	Launch checklist and handover	After 13

Open items

- Shopify content by the store team: FAQ page, case sizes, warranty text; then reconcile policy + full.
- Groq Developer tier (waiting) - else DeepInfra / Bedrock.
- Meta P2 verification, P3 billing, P5 real number via Coexistence + smb_message_echoes, P6 Live mode, P7 phone test + OWNER_FIRST=true.
- Small fix: a model number (e.g. 16610) can be read as a price limit by the router.
- Set COST_PER_* from real token counts after the paid plan.
- Delete the test voice clips after confirmation.

15. Where things are in the code

Folder / file	Contents
src/app/main.py, api/	FastAPI app, WhatsApp + Shopify webhooks, health, admin, /dev
src/app/worker.py	Inbound loop, cheap checks, voice, run_turn, outbox
src/app/agent/	graph.py (LangGraph + checkpointer), router.py (+cache), nodes.py, tools.py, composer, validator
src/app/catalog/	normalizer (aliases, brand short forms), search (trigram + browse), mirror
src/app/integrations/	shopify (GraphQL, auth, webhooks), whatsapp (payloads, sender, media), llm (chat factory, token count), speech, embeddings
src/app/voice/, language/, handoff/	voice-note pipeline, language detection, handoff + email notifier
src/app/rag/	policy ingest, chunking, vector search
src/app/security/	injection scan, card masking, secrets check, cool-down
src/app/owner/	owner-first presence and CLI
src/app/jobs/, sync/	reconcile, retention, Shopify sync worker
src/app/db/	SQLAlchemy models, repositories; alembic/ migrations
prompts/	router.md, composer.md, clarify / refusal / handoff templates (YAML)
config/aliases.yaml	Spelling variants and brand short forms
scripts/	register_webhooks.py, delete_customer.py, run_eval.py, bakeoff.py, loadtest.py, failure_drills.py, catalogue tools
tests/, evals/	2,139 tests; evals/: router_golden, product_qa, natural_qa, adversarial, rag_golden, voice_manifest
docs/	PROGRESS.md (running log), architecture.md, runbook.md, threat_model.md, security_checklist.md, test_cases.md, acceptance_criteria.md, adr/, eval_report.md, client_answers.md, cost and client PDFs, this overview

Golden rules for anyone continuing this project: never give the bot write access to Shopify; never let the model state a fact a tool did not return; keep secrets in .env only; run the tests and evals before every release; update docs/PROGRESS.md after each change.